

电子科技大学 计算机科学与工程学院

实验指导书

实验名称 7 段译码器的设计与实现

电子科技大学教务处制表

一、概述

时序逻辑电路的特点是，在任何时刻电路的稳定输出信号不仅与该时刻电路的输入信号（若有输入）有关，而且还与该电路的状态有关（与过去的输入有关）。通过本实验，学生将学习采用 Verilog 硬件描述语言设计数码管的显示接口逻辑电路，并在 FPGA 开发板进行测试，掌握时序逻辑电路的设计和调试方法。

7 段显示译码电路是将计算机中的 4 位 2 进制数转换成 7 段显示编码，在 CPU 应用中可以作为一个外设。使用 ISE 软件进行四位 7 段显示译码器的设计与实现，采用扫描显示时需要使用时钟，该时钟可以通过将系统时钟进行分频来获得。采用硬件描述语言 Verilog HDL 编程，生成流代码，下载到 FPGA 开发板上进行调试。本实验具体内容如下

1. 7 段显示编码模块的设计。
2. 使用 1 个数码管显示 4 位 2 进制数。
3. 时钟分频器的设计。
4. 六个数码管的扫描显示。

要求采用 Verilog 语言完成设计，设计工具为 Xilinx ISE 14.7 开发工具，FPGA 开发板是 Digilent 公司设计的 Anvyl 开发板，下载采用 Digilent Adept 工具。

二、实验原理：

在数字系统中，常常需要将数字、字母、符号的二进制代码翻译成人们习惯的形式，直观地显示出来，供人们读取或监视系统的工作情况。目前常用的有 7 段显示数码管。它是将 7 个发光二极管按一定的方式连接在一起，每段为一个发光二极管，7 段分别为 a、b、c、d、e、f、g，显示哪个字型，则对应段的发光二极管就发光。7 段显示数码管的结构如图 1 所示，图中的 dp 是小数点。

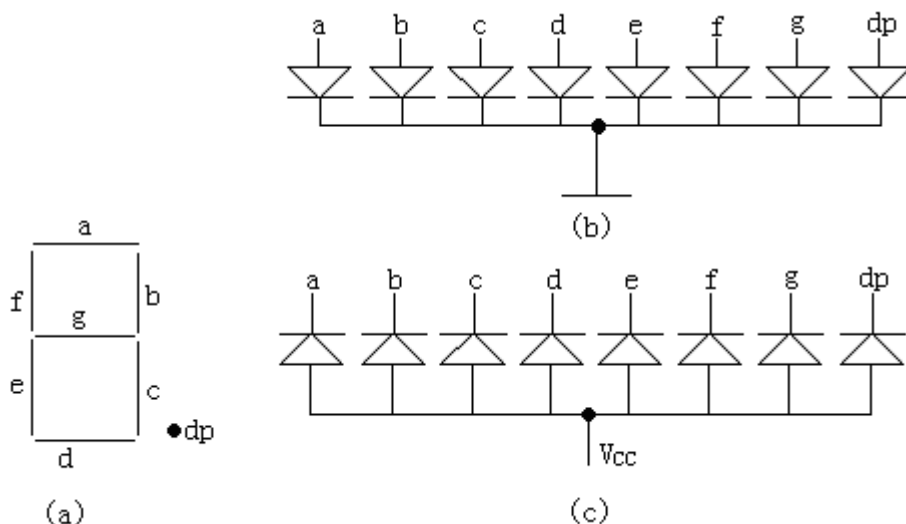


图 1 7 段显示数码管结构

7 段数码显示器分为共阴极和共阳极两种。所谓共阴是指数码管 7 个发光二极管的阴极都连在一起，接到地。发光二极管的阳极经过限流电阻接到 7 段译码器相应的输出端（译码器输出应为高电平有效）；所谓共阳是指数码管 7 个发光

二极管的阳极都连在一起，接到 V_{cc} ，而发光二极管的阴极经过限流电阻接到 7 段译码器相应的输出端（译码器输出应为低电平有效）。改变限流电阻可改变发光二极管的亮度。共阴和共阳数码管的结构如图 1（b）和（c）所示。

7 段数码管的驱动信号 $a\sim g$ 来自 4 线—7 线译码器。常用的中规模 4 线—7 线显示译码器有 74LS46~74LS49。当控制信号有效时， $A_3\sim A_0$ 输入 4 位二进制数， $a\sim g$ 输出端便有相应的输出，电路实现正常的译码。对于高有效的数码管，当译码输出为 1 时，对应数码管的字段就点亮。例如当 $A_3A_2A_1A_0=0101$ 时，只有 b 和 e 输出为 0，其余的都输出为 1， a 段、 c 段、 d 段、 f 段、 g 段点亮，显示数字“5”。

三、设计实现

1. 7 段显示译码器（显示和控制信号都是高有效）。

```
module SevenSegs(
    input  [3:0]    Digit ,
    input          EnableSegs ,
    output reg [6:0] Seg ) ;
    reg [6:0] A_G ;
    always @ (Digit or EnableSegs ) begin
        if (EnableSegs)
            case ( Digit )
// Segment patterns  abcdefg
                0: A_G = 7'b1111110; // 0
                1: A_G = 7'b0110000; // 1
                2: A_G = 7'b1101101; // 2
                3: A_G = 7'b1111001; // 3
                4: A_G = 7'b0110011; // 4
                5: A_G = 7'b1011011; // 5
                6: A_G = 7'b1011111; // 6
                7: A_G = 7'b1110000; // 7
                8: A_G = 7'b1111111; // 8
                9: A_G = 7'b1110011; // 9 (no 'tail')
                10: A_G = 7'b1110111; // A
                11: A_G = 7'b0011111; // b
                12: A_G = 7'b1001110; // C
                13: A_G = 7'b0111101; // d
                14: A_G = 7'b1001111; // E
                15: A_G = 7'b1000111; // F
                default: A_G = 7'bx;
            endcase
    end
```

```

        else A_G = 7'b0;
        Seg =
{A_G[0],A_G[1],A_G[2],A_G[3],A_G[4],A_G[5],A_G[6]} ;
    end
endmodule

```

2. 采用 1 个 7 段数码管显示四位二进制数。

Anvyl 开发板上数码管的显示和使能信号都是高有效，其测试代码如下

```

module OneSevenSegs_Anvyl(
    input  [3:0] Switch ,
    input      Btn0 ,
    output [6:0] Segs ,
    output      En    );

    assign En = ~Btn0 ; // Btn0 means for Disable

    SevenSegs
M1_SevenSegs( .Digit(Switch) , .EnableSegs(En) , .Seg(Segs) );

endmodule

```

Anvyl 开发板上的配置信息如下

```

#Switch
NET Switch[0] LOC= V5;
NET Switch[1] LOC= U4;
NET Switch[2] LOC= V3;
NET Switch[3] LOC= P4;

#Segment display Enable
NET EN LOC = M17; # 7-seg Enable 最右1个

##### 7-seg display #####
NET Segs[0] LOC=AA21; # 7-seg CA
NET Segs[1] LOC=AA22; # 7-seg CB
NET Segs[2] LOC=Y22; # 7-seg CC
NET Segs[3] LOC=N15; # 7-seg CD
NET Segs[4] LOC=AB19; # 7-seg CE
NET Segs[5] LOC=P20; # 7-seg CF
NET Segs[6] LOC=Y21; # 7-seg CG

#Button
Net Btn0    Loc= E6;

```

3. 时钟分频器的设计

```
module ClockDivider(  
    input      Reset ,  
    input      Clock ,  
    output reg DividedClock );  
parameter COUNTER = 32'hC350; // 50 000  
reg [32:0] clock_count;  
always @(posedge Clock or posedge Reset)  
    if (Reset == 1'b1)  
        clock_count <= {32{1'b0}};  
    else  
        begin  
            clock_count <= clock_count + 1;  
            if ( clock_count == COUNTER ) // CLK 为 100M 时,  
dividedClk 的周期 1ms , 高低各 0.5ms  
                begin  
                    DividedClock <= ~DividedClock;  
                    clock_count <= {32{1'b0}};  
                end  
        end  
end  
endmodule
```

4. 六个数码管的扫描显示

```
module SevenSegsController(  
    input      Reset ,  
    input      Clock ,  
    input [5:0] DisplayEnables ,  
    input [23:0] Data ,  
    output [7:0] SevenSegsAndPoint,  
    output [5:0] ShowOneofSix  
);  
reg [2:0] select_display;  
wire [3:0] four_digits;  
always @(posedge Clock or posedge Reset)  
    if (Reset == 1'b1)  
        select_display <= 3'b000;  
    else  
        begin  
            if (select_display == 5 )  
                select_display <= 3'b000;  
            else  
                select_display <= select_display + 1;  
        end  
end
```

```

    assign four_digits = (select_display[2:0] == 3'b000) ?
Data[3:0] :
                                (select_display[2:0] == 3'b001) ?
Data[7:4] :
                                (select_display[2:0] == 3'b010) ?
Data[11:8] :
                                (select_display[2:0] == 3'b011) ?
Data[15:12] :
                                (select_display[2:0] == 3'b100) ?
Data[19:16] :
                                Data[23:20];

    assign ShowOneofSix =
                                (select_display[2:0] == 3'b000 &
DisplayEnables[0] == 1'b1) ? 6'b000001 :
                                (select_display[2:0] == 3'b001 &
DisplayEnables[1] == 1'b1) ? 6'b000010 :
                                (select_display[2:0] == 3'b010 &
DisplayEnables[2] == 1'b1) ? 6'b000100 :
                                (select_display[2:0] == 3'b011 &
DisplayEnables[3] == 1'b1) ? 6'b001000 :
                                (select_display[2:0] == 3'b100 &
DisplayEnables[4] == 1'b1) ? 6'b010000 :
                                (select_display[2:0] == 3'b101 &
DisplayEnables[5] == 1'b1) ? 6'b100000 :
                                6'b000000;

    SevenSegs
M1_SevenSegs(.Digit(four_digits) , .EnableSegs(DisplayEnab
les[select_display]) , .Seg(SevenSegsAndPoint[6:0])) ;

    assign SevenSegsAndPoint[7] = 1'b0;

endmodule

```

```

module SixSevenSegs_Anvy1(
    input      Clock,
    input  [5:0] Switch,
    output [5:0] En ,
    output [7:0] Segs );
    wire      divided_clock;
    wire  [23:0] data ;
    assign data = 24'h543210 ;
    ClockDivider
M1_ClockDivider(.Reset(1'b0), .Clock(Clock), .DividedClock
(divided_clock));
    SevenSegsController M2_SevenSegsController(
        .Reset(1'b0), .Clock(divided_clock), .DisplayEn
ables(Switch),
        .Data(data), .SevenSegsAndPoint(Segs), .ShowOneofSix(E
n));
endmodule

```

注意 Anvy1 开发板上六个数码管的排列次序比较特殊，六个数码管分为 3 组，每组 2 个数码管，组内序号低的在左，序号高的在右。为便于使用，在定义使能信号的引脚时可以用下面的次序。

```

// dj : change define sequence
NET En[1] LOC = P16; # 7-seg AN0
NET En[0] LOC = M17; # 7-seg AN1
NET En[3] LOC = N16; # 7-seg AN2
NET En[2] LOC = P19; # 7-seg AN3
NET En[5] LOC = AA20; # 7-seg AN4
NET En[4] LOC = AB21; # 7-seg AN5

```

四、实验调试

Anvy1 开发板有 6 个 7 段数码管，可以通过拨动 SW5-SW0 来控制每个数码管是否显示。

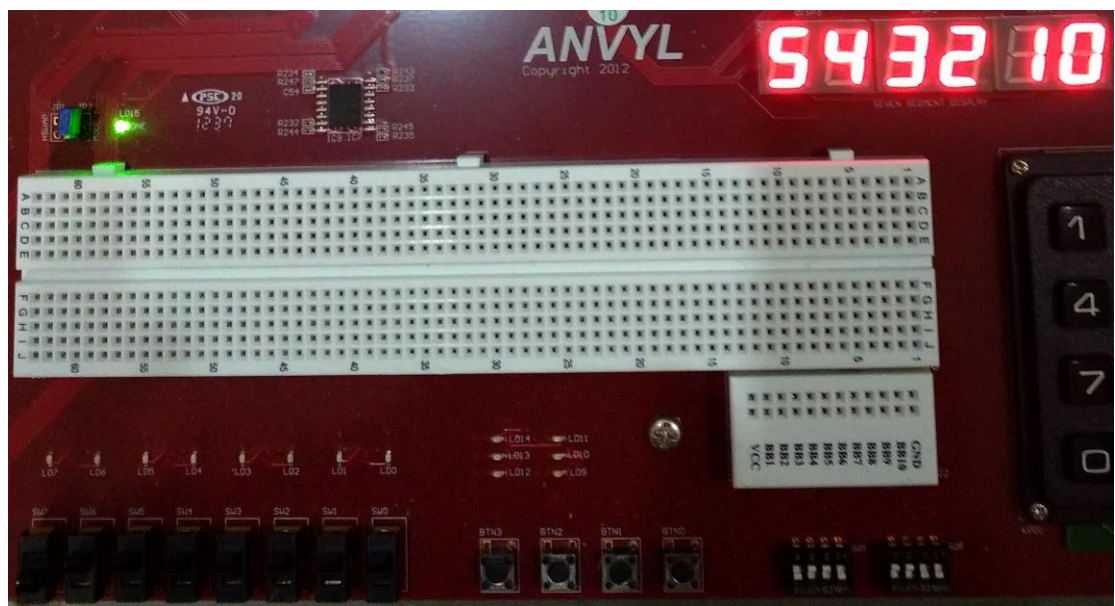


图 2 Anvyl 开发板数码管的显示